Checklist de déploiement kiosk

Ce que le relecteur d'admission cert doit s'attendre à trouver sur la machine de test au démarrage de la clé USB.

Couche matériel

- ☐ Penetek A3216 M82 ou Advantech USC-360J (ou terminal x86_64 fanless équivalent avec écran tactile PCAP 15.6" 1920×1080).
- ☐ Imprimante thermique Epson TM-m30III (USB ou TCP).
- ☐ Tiroir-caisse RJ11 raccordé à l'imprimante.
- ☐ Credentials du FDM virtuel (zwartedoos.be) configurés dans `/etc/moyun-pos/pos.json::fdm_url`.

Couche OS

- ☐ Debian 12 minimal (sans environnement de bureau).
- ☐ Compositeur Cage Wayland - lance automatiquement `moyun-pos` en plein écran au boot.
- ☐ Service systemd `moyun-pos.service` activé.
- ☐ Heure système synchronisée via chrony (les horodatages fiscaux corrects sont une exigence cert).
- ☐ Pas de serveur SSH (appliance scellée). Accès console uniquement sur le terminal physique.

Couche application

- ☐ Build cert de `moyun-pos` installée dans `/opt/moyun-pos/`. Compilée avec : `cargo build --release --no-default-features --features cert`.
- ☐ Binaire `pos-fdm-mock` dans `/opt/moyun-pos/bin/pos-fdm-mock` (utilisé pendant la démo d'admission si le FDM virtuel est injoignable).
- ☐ Base de données de seed dans `/var/lib/moyun-pos/pos.db` avec : - Configuration Tokyo Sushi Bar (nom, n° TVA, adresse). - Menu complet (categories, menu_items, modifier_groups, modifiers). - Utilisateurs de test correspondant à la liste INSZ dans `docs/20250916_UC_BasicSet_POS_FDM_1.7/UC_TestSetup_20240`
- ☐ IP LAN de l'imprimante configurée dans `pos.json::printer_address`.

Documentation dans la clé USB du dossier

- ☐ `DOCUMENT_PROCESSUS_TECHNIQUE_FR.md`
- ☐ `FLUX_DONNEES_FR.md`
- ☐ `MAPPING_CONFORMITE_FR.md`
- ☐ `STRATEGIE_COMPTEURS_FR.md`
- ☐ `COUVERTURE_FIXTURES_FR.md` + `COUVERTURE_RAPPORTS_FR.md` + `COUVERTURE_ADVANCED_SET_FR.md`
- ☐ `FLUX_REMBOURSEMENT_FR.md`
- ☐ `DECISION_PEPPOL_AP_FR.md`
- ☐ `LIGNES_PRODUITS_FR.md`
- ☐ `LISTE_DISTRIBUTEURS_FR.md`
- ☐ `RAPPORT_TESTS_BASELINE_FR.md`
- ☐ `DOSSIER_SOUMISSION_FR.md`
- ☐ Formulaire de demande de certification signé (PDF + copie scannée signée).
- ☐ `JOURNAL_TESTS_PRE_SOUMISSION_FR.md` (rempli après la répétition de Phase 11).

Commandes de vérification (sur la machine de test)

L'image kiosk de production embarque **sans code source ni toolchain Rust** - uniquement le binaire signé, la base SQLite et le service systemd. Les commandes de vérification ci-dessous s'exécutent sur la **machine de build du développeur** (un poste de travail séparé), contre l'arbre source qui a produit le binaire, pas sur le kiosk.

```
# Sur la MACHINE DE BUILD (arbre source présent), avant la production
# de l'image kiosk :
cd ~/moyun-pos                          # le dépôt source

# 1) Confirmer que le binaire cert ne contient aucune intégration
#    tierce désactivée (symboles résiduels attendus = 0).
cargo build --release --no-default-features --features cert
nm target/release/moyun-pos | grep -c pos_hubrise         # → 0

# 2) Confirmer que fiscal_events est append-only (pas de DELETE/UPDATE
#    dans le code).
grep -r "DELETE FROM fiscal_events\|UPDATE fiscal_events" \
  src-tauri/ crates/                                              # → aucun match

# 3) Lancer la suite de tests hors-ligne.
cargo test --workspace --no-fail-fast                             # → tout vert

# 4) Smoke test contre le FDM virtuel (zwartedoos.be).
MOYUN_STAGING_FDM_URL=https://... \
  cargo test --package pos-fiscal --test e2e_staging -- --ignored
# → tous les UC passent avec signatures vérifiées

# 5) Calculer le SHA-1 du binaire cert pour le formulaire.
sha1sum target/release/moyun-pos
```

Sur le **kiosk lui-même**, le relecteur peut vérifier que la version en cours d'exécution correspond au hash du dossier :

```
sha1sum /opt/moyun-pos/moyun-pos
# → doit correspondre à la valeur déclarée dans le formulaire cert
```

Script de démonstration pour le rendez-vous d'admission

1. Boot du kiosk ; montrer Cage qui lance moyun-pos en plein écran.
2. Login avec INSZ ; montrer l'événement fiscal **S** work-in archivé.
3. Ouvrir Table 4 ; ajouter trois articles ; envoyer en cuisine - événement **P** + ticket cuisine.
4. Split de la table 2/1 ; payer un côté en espèces, l'autre en carte - **N** ×2.
5. Réimprimer le ticket espèces - événement **C** avec `fdmRef`.
6. Déclencher un rapport X - événement **R**.
7. Déclencher la clôture journalière Z - événements **R**+**R**, tiroir ouvert.
8. Pointage sortie - événement **S**.
9. Inspecter `fiscal_events` dans sqlite - compteurs strictement monotones par label ; chaque ligne a un `fdm_response_json` non-null.
10. Démontrer le blocage en cas de FDM déconnecté en tuant le mock FDM en plein milieu d'un flux.